

Operating Systems Lab Mini Project: Concurrent Ride-Sharing Dispatch System

Parth Malhotra
Roll No: BT2024197

April 30, 2026

1 Problem Statement

In the modern era of on-demand transportation services, companies like Ola and Uber rely on highly distributed, real-time systems to connect millions of riders with available drivers. The core challenge of such a system is not just connecting two people, but doing so while managing thousands of concurrent requests, ensuring fair pricing through dynamic supply-demand metrics, and maintaining absolute data integrity in a distributed environment.

For this project, I have developed a Concurrent Ride-Sharing Dispatch System that simulates these complex operations. The system must address the following core functionalities:

- **High Concurrency:** The system must handle multiple riders and drivers interacting with a single central server simultaneously without crashing or missing requests.
- **Real-Time Matchmaking:** Implementing a Nearest Driver algorithm that calculates Euclidean distances across a coordinate grid in real-time.
- **Dynamic Surge Pricing:** Just like real-world apps, the system monitors driver availability and automatically spikes fares during shortages to balance the ecosystem.
- **Role-Based Ecosystem:** Providing distinct interfaces and capabilities for Riders (booking), Drivers (service delivery), and Administrators (system oversight).

To build this, I utilized advanced Operating System primitives including POSIX threads, Shared Memory, Semaphores, and Advisory File Locking to ensure the system is as robust as a production-grade application.

2 Detailed Implementation of Core Concepts

2.1 4.1 Role-Based Authorization (RBAC)

I implemented a strict RBAC system to ensure that users can only perform actions relevant to their identity. This is critical for preventing unauthorized access to administrative tools or driver earnings.

- **Data Structure:** I used an enum in `structs.h` to define `ROLE_RIDER`, `ROLE_DRIVER`, and `ROLE_ADMIN`.
- **Implementation:** During authentication, the server reads the role from the `users.dat` file. Every incoming request is then passed through a Gatekeeper loop in `server_main.c`. I used a switch statement that validates the packet type against the user's role (e.g., a Rider cannot send a `MSG_ADMIN_ACTION` packet). If the validation fails, the server rejects the request immediately.

2.2 4.2 File Locking (Advisory Locking)

Data persistence in a concurrent system is dangerous without synchronization. I used POSIX Advisory Locking to manage access to the system ledger and user database.

- **The `fcntl()` Mechanism:** I used the `flock` struct and `fcntl()` system call to implement granular locks.
- **Write Locks:** In the `log_trip()` function, I applied an exclusive `F_WRLCK`. This ensures that if ten drivers finish trips at once, the OS serializes their writes to `trip_history.txt`, preventing Lost Updates.
- **Read/Write Balance:** For the Admin dashboard, I used shared read locks (`F_RDLCK`) allowing multiple admins to generate reports simultaneously, but blocking them if a write operation is in progress, thereby preventing Dirty Reads.

2.3 4.3 Concurrency Control (Threads, Mutexes, and Semaphores)

Handling hundreds of simultaneous clients requires sophisticated synchronization to avoid race conditions.

- **Thread Management:** I used `pthread_create` to spawn a dedicated worker thread for every client connection. I utilized `pthread_detach` to ensure that resources are automatically reclaimed by the OS when a user logs out.
- **Mutexes:** I implemented `driver_mutex` to protect the Shared Memory driver grid. Since multiple threads might try to book the same driver, the mutex ensures that the search and status-update (Offline to On-Trip) is an atomic operation.
- **Semaphores:** I used a POSIX Named Semaphore (`sem_t`) as a thread-safe counter for the global driver pool. This allows the matchmaking logic to instantly know if the supply is zero without having to scan the entire grid.

2.4 4.4 Data Consistency

Consistency is the cornerstone of any dispatch system. I focused on preventing three major architectural failures:

- **Race Conditions:** By using mutexes around the driver matching algorithm, I ensured that a driver can only be assigned to one rider.

- **Lost Updates:** In the `handle_admin_action` function, I used file locks to ensure that when an admin bans a user, the change is written completely to the disk before any other process can read it.
- **Dirty Reads:** By using read-locks in the history viewing logic, I ensured that riders never see a trip record that is only partially written to the file.

2.5 4.5 Socket Programming (Network Layer)

The system is built on a distributed TCP Client-Server Architecture.

- **The Server:** I implemented a robust listener in `server_main.c` using `AF_INET` sockets. I also used `SO_REUSEADDR` to allow for immediate server restarts during testing.
- **Communication Protocol:** I designed a custom binary protocol using the `MessagePacket` struct. This ensures that the server can parse different request types (Auth, Location, Ride Requests) by simply looking at the packet header, making the network communication extremely efficient.

2.6 4.6 Inter-Process Communication (IPC)

I utilized two distinct forms of IPC to demonstrate a deep understanding of Operating System memory management:

- **Shared Memory:** This is the system's High-Speed Backbone. I created a shared memory segment for the Driver Grid. This allows the `surge_calc` process (an external process) to monitor the server's state in real-time without the latency of socket communication.
- **Signal Handling:** I implemented `signal(SIGINT, cleanup_and_exit)` to trap terminal interrupts. This ensures that even if the system is forcefully closed, it cleans up all IPC resources (unlinking SHM and Semaphores) to maintain system hygiene.

3 System Execution and Output

3.1 Server Initialization

The server starts by initializing all IPC resources and opening the port for connections.

```

parthmalhotra@LAPTO x parthmalhotra@LAPTOI x parthmalhotra@LAPTOI x parthmalhotra@LAPTOI x parthmalhotra@LAPTOI x + - X
Live Metrics:
  Drivers Available : 1
  Drivers On Trip   : 0
  Drivers Offline   : 0

Current Surge: 1.5x (Elevated Demand)
Monitoring Shared Memory Grid instantly...
^C
parthmalhotra@LAPTOP-IQTIEU8S:/mnt/c/Users/Parth Malhotra/OneDrive/Desktop/OS_Project$ make all
mkdir -p bin || type nul >> nul
mkdir -p data || type nul >> nul
gcc -Wall -Wextra -pthread -g src/server/server_main.c -o bin/server -lrt
gcc -Wall -Wextra -pthread -g src/client/driver.c -o bin/driver -lrt
gcc -Wall -Wextra -pthread -g src/client/rider.c -o bin/rider -lrt
gcc -Wall -Wextra -pthread -g src/client/admin.c -o bin/admin -lrt
gcc -Wall -Wextra -pthread -g src/ipc/surge_calc.c -o bin/surge_calc -lrt
parthmalhotra@LAPTOP-IQTIEU8S:/mnt/c/Users/Parth Malhotra/OneDrive/Desktop/OS_Project$ ./bin/server
IPC and Synchronization primitives initialized.
Server is up and running on port 8080.
New client connected on Socket 5.
User 'admin!' authenticated.
New client connected on Socket 6.
User 'driver1' authenticated.
New client connected on Socket 7.
User 'rider1' authenticated.
New client connected on Socket 8.
User 'driver1' authenticated.
Searching for nearest driver for Rider 3...
Trip finished for Rider 3. Fare: $87

```

Figure 1: Main Server bootstrapping IPC and Network Sockets

3.2 Rider Workflow

The rider logs in, chooses a destination, and waits for the server to find a nearby driver.

```

parthmalhotra@LAPTOI x parthmalhotra@LAPTOI x parthmalhotra@LAPTOI x parthmalhotra@LAPTOI x parthmalhotra@LAPTOI x + - X
PS C:\Users\Parth Malhotra> wsl
parthmalhotra@LAPTOP-IQTIEU8S:/mnt/c/Users/Parth Malhotra$ cd OneDrive/Desktop/OS_Project
parthmalhotra@LAPTOP-IQTIEU8S:/mnt/c/Users/Parth Malhotra/OneDrive/Desktop/OS_Project$ ./bin/rider
--- Welcome to the Ride-Sharing System (Rider) ---
Username: rider1
Password: riderpass1
Successfully logged in! Your ID is 3

Main Menu:
1. Book a Ride
2. View My History
3. Logout
Enter choice: 1
Enter Your Current X and Y: 1 2
Enter Your Destination X and Y: 3 4
Looking for a driver nearby...
Matched with Driver ID 2! They are coming to pick you up.

Main Menu:
1. Book a Ride
2. View My History
3. Logout
Enter choice: 2
--- Your Rides ---
-> Trip to (0,0) | Fare: $37
-> Trip to (2,2) | Fare: $62
-> Trip to (30,40) | Fare: $537
-> Trip to (20,30) | Fare: $287
-> Trip to (20,20) | Fare: $287
-> Trip to (3,4) | Fare: $87

Main Menu:
1. Book a Ride
2. View My History
3. Logout
Enter choice: |

```

Figure 2: Rider authentication and successful ride matching

3.3 Driver Ride Offer

The driver receives a Push Notification through a background listener thread when a ride is offered.

```
parthmalhotra@LAPTOI x parthmalhotra@LAPTOI x parthmalhotra@LAPTOI x parthmalhotra@LAPTOI x parthmalhotra@LAPTOI x + - X
6. Reject Last Offer
7. Exit
Choice:
!!! New Ride Request !!!
From: (1, 2) To: (3, 4)
Go to menu and choose '5' to Accept or '6' to Reject.
Choice: 5
You accepted the ride!

Driver Menu:
1. Go Online
2. Go Offline
3. Update Location (X,Y)
4. View My Earnings
5. Accept Last Offer
6. Reject Last Offer
7. Exit
Choice: 4

--- My Earnings ---
-> Trip From (0,0) | Earned: $37
-> Trip From (1,1) | Earned: $62
-> Trip From (10,20) | Earned: $537
-> Trip From (10,20) | Earned: $287
-> Trip From (10,10) | Earned: $287
-> Trip From (1,2) | Earned: $87
TOTAL EARNINGS: $1297

Driver Menu:
1. Go Online
2. Go Offline
3. Update Location (X,Y)
4. View My Earnings
5. Accept Last Offer
6. Reject Last Offer
7. Exit
Choice:
```

Figure 3: Background listener catching a ride offer while in menu

3.4 Administrator Live Grid

The admin uses Shared Memory to spy on the live status of all drivers across the system.

```
parthmalhotra@LAPTOI x parthmalhotra@LAPTOI x parthmalhotra@LAPTOI x parthmalhotra@LAPTOI x parthmalhotra@LAPTOI x + - X
Admin Menu:
1. View Live Driver Grid
2. View All Trip Records
3. Calculate Total Earnings
4. Lock/Unlock a User Account
5. Exit
Choice: 1

--- Global Driver Grid ---
ID 2 | Status: 1 | At: (3, 4)

Admin Menu:
1. View Live Driver Grid
2. View All Trip Records
3. Calculate Total Earnings
4. Lock/Unlock a User Account
5. Exit
Choice: 2

--- Full System History ---
# Trip Ledger (Protected via fcntl)
# Format: TripID RiderID DriverID StartX StartY EndX EndY Fare
TRIP 3 2 0 0 0 0 37
TRIP 3 2 1 1 2 2 62
TRIP 3 2 10 20 30 40 537
TRIP 3 2 10 20 30 30 287
TRIP 3 2 10 10 20 20 287
TRIP 3 2 1 2 3 4 87

Admin Menu:
1. View Live Driver Grid
2. View All Trip Records
3. Calculate Total Earnings
4. Lock/Unlock a User Account
5. Exit
Choice: |
```

Figure 4: Admin dashboard viewing real-time shared memory grid

3.5 Surge Pricing Monitor

A standalone IPC process that monitors demand and adjusts fare multipliers dynamically.

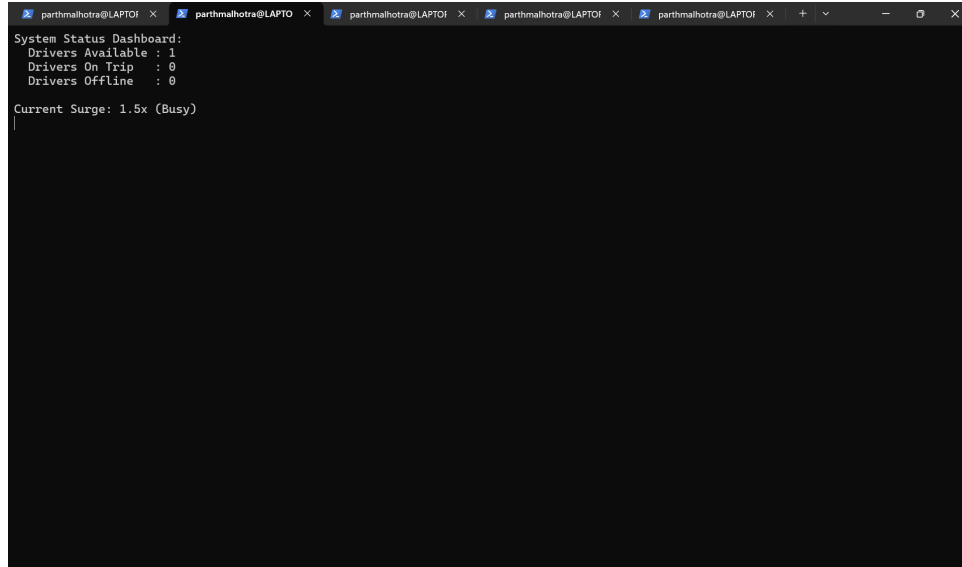


Figure 5: Surge Pricing Calculator adjusting multipliers in real-time

4 Challenges Faced and Solutions

4.1 Challenge 1: Race Conditions in Driver Matching

Problem: During high-load testing, I noticed that two riders could occasionally be matched with the same driver if their requests arrived simultaneously.

Solution: I wrapped the entire Search-and-Claim logic in a mutex lock. This made the matchmaking process atomic, meaning the server can only process one rider's search at a time, guaranteeing uniqueness.

4.2 Challenge 2: Resource Leaks in IPC

Problem: If the server was terminated using Ctrl+C, the shared memory and named semaphores persisted in the OS, causing File Exists errors on the next run.

Solution: I implemented a SIGINT signal handler that traps the exit signal and calls a cleanup routine to unlink all POSIX IPC resources, ensuring a clean slate for every run.

4.3 Challenge 3: Asynchronous Notifications

Problem: Drivers needed to receive ride offers even while they were blocked by a `scanf()` call in their main menu.

Solution: I implemented a dedicated Background Listener Thread in the driver client. This thread constantly waits for server messages while the main thread handles user input, solving the blocking issue entirely.